

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Архитектура вычислительных систем

ОТЧЕТ
к лабораторной работе № 1
на тему «Защищенный режим 32-разрядных процессоров»

Выполнил:
студент гр. 153503
Кахновский Е.С.

Проверил:
Калиновская А.А.

Минск 2023

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Выводы	6
Список использованных источников	7
Приложение А (обязательное) Листинг кода.....	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью выполнения данной лабораторной работы является изучение особенности защищенного режима процессора, получение практических навыков по программированию переключения процессора из реального в защищенный режим и обратно.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

32-разрядные процессоры Intel 80386, 80486 и Pentium с точки зрения рассматриваемых в данном разделе вопросов имеют аналогичные средства, поэтому для краткости в тексте используется термин "процессор i386", хотя вся информация этого раздела в равной степени относится к трем моделям процессоров фирмы Intel.

Процессор i386 имеет два режима работы - реальный (real mode, R-Mode) и защищенный (protected mode, P-Mode).

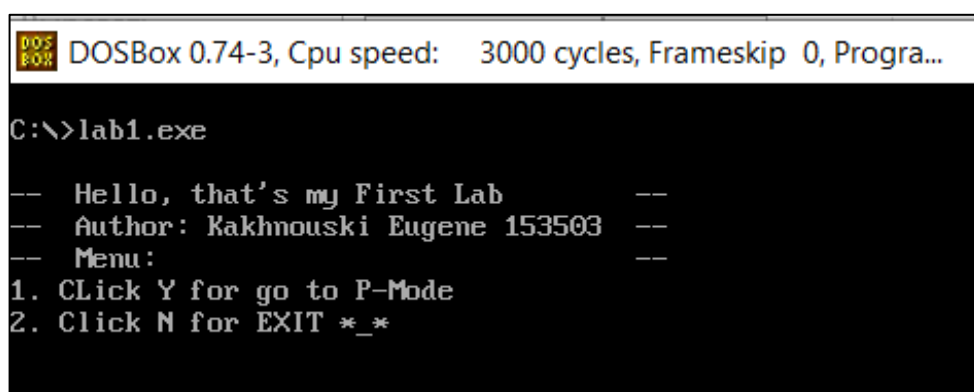
Реальный режим – это режим, в котором процессор работает как быстрый процессор 8086, но позволяет пользоваться большинством своих технологий (MMX / SSE / SSE2, 32-разрядными регистрами общего назначения, регистрами управления и отладки и пр.). После аппаратного сброса процессор переходит в этот режим и начинает выполнять программную инициализацию из BIOS-а. Реальный режим в современных процессорах предназначен для запуска компьютера и подразумевается, что операционная система будет работать в защищённом режиме (поэтому оптимизация по производительности для процессоров IA-32 производится для защищённого режима).

В реальном режиме не доступны основные достоинства процессора – виртуальная память, мультизадачность, уровни привилегий, работа с кэшами, буферами TLB, буфером ветвлений и некоторыми другими технологиями, обеспечивающими высокую производительность.

Как утверждает Intel, это "родной" (native) режим 32-разрядного процессора. В защищённый режим процессор надо переводить специальными операциями над системными регистрами и войти в этот режим процессор может только из реального режима. При работе в защищённом режиме процессор контролирует практически все действия программ и позволяет разделить операционную систему, драйвера и прикладные программы разными уровнями привилегий. Благодаря этому ОС может ограничить области памяти, выделяемой программам, запретить или разрешить ввод/вывод по любым адресам, управлять прерываниями и многое другое. При попытке программы выйти за допустимый диапазон адресов памяти, выделенной ей, либо при обращении к "запрещённым" для неё портам процессор будет генерировать исключения – специальный тип прерываний. Грамотно оперируя исключениями, операционная система может контролировать действия программ, организовать систему виртуальной памяти, мультизадачность и другие программные технологии.

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

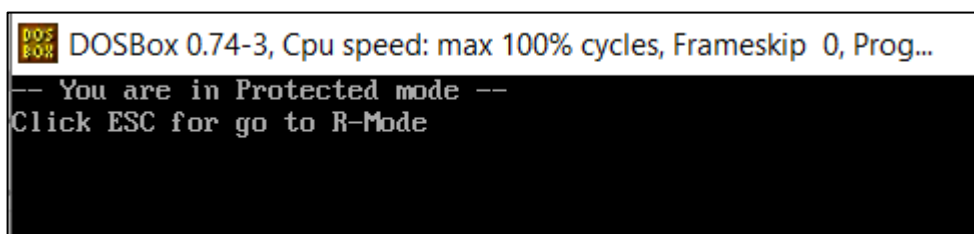
В рамках выполнения лабораторной работы была создана приложение, которое осуществляет переключение процессора между защищенным режимом и реальным режимом. Приложение также включает вывод сообщения на экран, после успешного переключения в защищенный режим работы процессора и обратно (рисунок 3.1, рисунок 3.2).



```
DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Progra...
C:\>lab1.exe

-- Hello, that's my First Lab --
-- Author: Kakhnouski Eugene 153503 --
-- Menu: --
1. CLick Y for go to P-Mode
2. Click N for EXIT *_*
```

Рисунок 3.1 – Результат работы меню программы



```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Prog...
-- You are in Protected mode --
Click ESC for go to R-Mode
```

Рисунок 3.2 – Результат успешного перехода в защищенный режим

ВЫВОДЫ

В результате выполнения этой лабораторной работы была разработана программа с использованием эмулятора DOSBox и Turbo Assembler (TASM), которая выполняет переключение процессора между защищенным и реальным режимом работы. При этом программа отображает сообщение на экране об успешном переходе между режимами и сообщение из самих режимов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Юрий А. Уровни защиты. – СПб: Питер, 2000. – 158 с.
- [2] [Электронный ресурс]. – Режим доступа:
https://dfe.petsu.ru/x86/env_1.shtml – Дата доступа 08.10.2023

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

Листинг 1 – Файл lab1.asm

```

printStr macro str
    push ax
    push dx

    mov ah,09h ; íîîäð òóîëöèè DOS
    lea dx,str ; Ñîäóâîèâ str â DX
    int 21h ; 21 îðäðóââîèâ

    pop dx
    pop ax
endm

.386P
.MODEL LARGE
;Структуры данных
S_DESC struc ;Структура сегментного
    дескриптора
        LIMIT dw 0 ;Лимит сегмента (15:00)
        BASE_L dw 0 ;Адрес базы, младшая часть
        (15:0)
        BASE_M db 0 ;Адрес базы, средняя часть
        (23:16);Флаги уровней доступа сегментов
        ACS_PRESENT EQU 10000000B ;PXXXXXXX - бит присутствия,
        сегмент присутствует в оперативной памяти
        ACS_CSEG EQU 00011000B ;XXXXIXXX - тип сегмента, для
        данных = 0, для кода 1
        ACS_DSEG EQU 00010000B ;XXSXSSXX - бит сегмента,
        данный объект сегмент(системные объекты могут быть не сегменты)
        ACS_READ EQU 00000010B ;XXXXXXRX - бит чтения,
        возможность чтения из другого сегмента
        ACS_WRITE EQU 00000010B ;XXXXXXWX - бит записи, для
        сегмента данных разрешает запись
        ACS_CODE = ACS_PRESENT or ACS_CSEG ;AR сегмента кода
        ACS_DATA = ACS_PRESENT or ACS_DSEG or ACS_WRITE;AR сегмента данных
        ACS_STACK= ACS_PRESENT or ACS_DSEG or ACS_WRITE;AR сегмента стека
        ACS_INT_GATE EQU 00001110B
        ACS_TRAP_GATE EQU 00001111B ;XXXXXSICR - сегмент,
        подчиненный сегмент кода, доступен для чтения
        ACS_IDT EQU ACS_DATA ;AR таблицы IDT
        ACS_INT EQU ACS_PRESENT or ACS_INT_GATE
        ACS_TRAP EQU ACS_PRESENT or ACS_TRAP_GATE
        ACS_DPL_3 EQU 01100000B ;X<DPL,DPL>XXXXX - привелегии
        доступа, доступ может получить любой кодSTART:
        mov ax,DATA ;Инициализация сегментных
        регистров
        mov ds,ax
        mov es,ax
        printStr NEW_LINE_MSG

```

```

    printStr START_MSG
    printStr NEW_LINE_MSG
    printStr AUTHOR_MSG
    printStr NEW_LINE_MSG
    printStr MENU_MSG
    printStr NEW_LINE_MSG
    printStr TO_PROTECT_MSG
    printStr NEW_LINE_MSG
    printStr EXIT_MSG
    printStr NEW_LINE_MSG

ANSWER:
    mov ah, 8h
    int 21h                                ;Ожидание подтверждения
    cmp al, 'y'
    je ENABLE_A20
    cmp al, 'n'
    je END_PROG
    jmp ANSWER

ENABLE_A20:                                ;Открыть линию A20
    in al, 92h
    or al, 2                                ;Установить бит 1 в 1
    out 92h, al

BUFFER_OUTPUT proc near                    ;Процедура вывода текстового
буфера, оканчивающегося 0
    push es
    pushad
    mov ax, TEXT_DESC                      ;Поместить в es селектор
текста
    mov es, ax
OUTPUT_LOOP:                               ;Цикл по выводу буфера
    lodsb
    or al, al
    jz OUTPUT_EXIT                         ;Если дошло до 0, то конец
выхода
    stosb
    inc edi
    jmp OUTPUT_LOOP
OUTPUT_EXIT:                               ;Выход из процедуры вывода
    popad
    pop es
    ret

BUFFER_OUTPUT ENDP
SIZE_CODE_PM = ($ - CODE_PM_BEGIN)
CODE_PM ENDS
;Сегмент данных реального/защищенного режима
DATA segment para use16                   ;Сегмент данных
реального/защищенного режима
DATA_BEGIN = $
    ;GDT - глобальная таблица дескрипторов
    GDT_BEGIN = $
    GDT label word                        ;Метка начала GDT
    GDT_0 S_DESC <0,0,0,0,0,0>
    GDT_GDT S_DESC <GDT_SIZE-1,,,ACS_DATA,0,>
    GDT_CODE_RM S_DESC <SIZE_CODE_RM-1,,,ACS_CODE,0,>

```

```

GDT_DATA      S_DESC <SIZE_DATA-1,,,ACS_DATA+ACS_DPL_3,0,>
GDT_STACK     S_DESC <1000h-1,,,ACS_DATA,0,>
GDT_TEXT      S_DESC <2000h-1,8000h,0Bh,ACS_DATA+ACS_DPL_3,0,0>
GDT_CODE_PM   S_DESC <SIZE_CODE_PM-1,,,ACS_CODE+ACS_READ,0,>
GDT_IDT       S_DESC <SIZE_IDT-1,,,ACS_IDT,0,>
GDT_SIZE      = ($ - GDT_BEGIN)                ;Размер GDT      CODE_RM_DESC =
(GDT_CODE_RM - GDT_0)
DATA_DESC     = (GDT_DATA - GDT_0)
STACK_DESC    = (GDT_STACK - GDT_0)
TEXT_DESC     = (GDT_TEXT - GDT_0)
CODE_PM_DESC  = (GDT_CODE_PM - GDT_0)
IDT_DESC      = (GDT_IDT - GDT_0)
;IDT - таблица дескрипторов прерываний
IDTR          R_IDTR <SIZE_IDT,0,0>              ;Формат регистра IDTR
IDT label     word                               ;Метка начала IDT
IDT_BEGIN     = $
IRPC          N, 0123456789ABCDEF
                IDT_0&N I_DESC <0, CODE_PM_DESC,0,ACS_TRAP,0>          ; 00...0F
ENDM
IRPC          N, 0123456789ABCDEF
                IDT_1&N I_DESC <0, CODE_PM_DESC, 0, ACS_TRAP, 0>        ; 10...1F
ENDM
IDT_20        I_DESC <0, CODE_PM_DESC,0,ACS_INT,0>
IDT_KEYBOARD  I_DESC <0, CODE_PM_DESC,0,ACS_INT,0>          ;IRQ 1 -
прерывание клавиатуры
IRPC          N, 23456789ABCDEF
                IDT_2&N          I_DESC <0, CODE_PM_DESC, 0, ACS_INT, 0> ; 22...2F
ENDM
SIZE_IDT      =          ($ - IDT_BEGIN)
NEW_LINE_MSG  db 0Ah, 0Dh, '$'
START_MSG     db "-- Hello, that's my First Lab          --$"
AUTHOR_MSG    db "-- Author: Kakhnouski Eugene 153503    --$"
MENU_MSG      db "-- Menu:                                --$"
EXIT_MSG      db "2. Click N for EXIT *_*$"
TO_PROTECT_MSG db "1. CClick Y for go to P-Mode$"
ESC_MSG       db "Click ESC for go to R-Mode", 0
MSG_HELLO     db "- Press 'y' to go to the protected
mode",13,10,"$"
MSG_HELLO_PM  db "-- You are in Protected mode --",0
MSG_HELLO_RM  db "-- You are in Real mode --",13,10,"$"
MSG_KEYBOARD  db "-- Press 'ESC' to come back to the real mode",0
MSG_EXC       db "exception: XX",0
MSG_EXIT      db "- Press 'N' to exit",13,10,"$"
SIZE_DATA     = ($ - DATA_BEGIN)                ;Размер сегмента данных
DATA          ends
;Сегмент стека реального/защищенного режима
STACK_A segment para stack
                db 1000h dup(?)
STACK_A       ends
end START

```